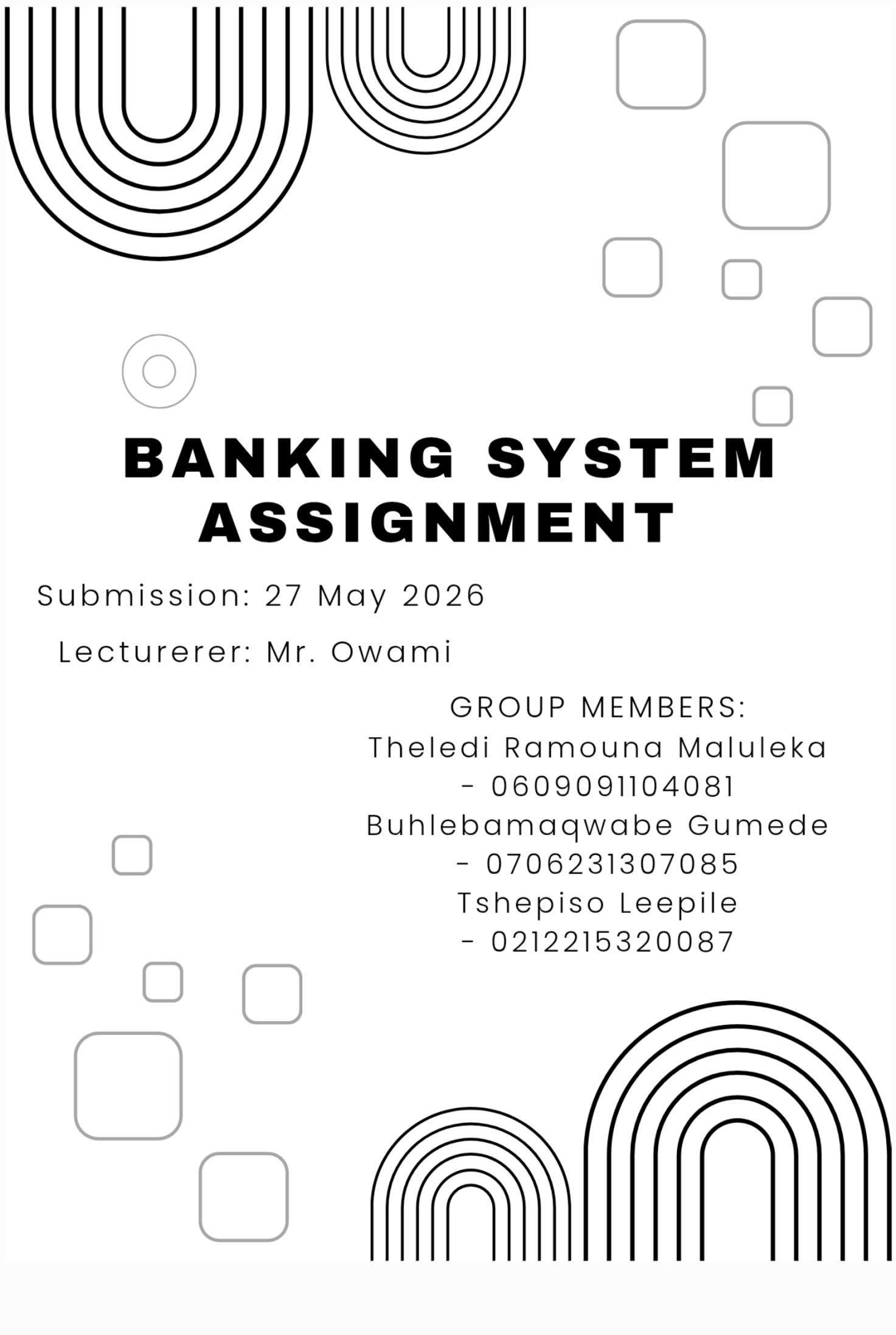




BANKING SYSTEM ASSIGNMENT

Submission: 27 May 2026

Lecturer: Mr. Owami

GROUP MEMBERS:

Theledi Ramouna Maluleka

- 0609091104081

Buhlebamaqwabe Gumede

- 0706231307085

Tshepiso Leepile

- 0212215320087

Table of Contents

INTRODUCTION	3
Purpose and scope of the system	3
Background and motivation	3
OBJECTIVES OF A JAVA BASED BANKING SYSTEM	3
SYSTEM OVERVIEW	4
HIGH LEVEL DESCRIPTION OF THE BANKING SYSTEM	4
LIST OF CORE FUNCTIONALITIES	5
TECHNOLOGIES AND TOOLS USED	5
DATABASE DESIGN	6
ENTITY RELATIONSHIP DIAGRAM (ERD)	6
EXPLANATION OF THE DATABASE TABLES.....	7
PRIMARY AND FOREIGN KEY JUSTIFICATION.	7
NORMALIZATION EXPLAINATON	9
APPLICATION FEATURES DOCUMENTATION	9
FEATURE LIST WITH DESCRIPTION	9
Screenshots of running application	11
INPUT/OUTPUT DESCRIPTIONS	22
ERROR HANDLING IN THE BANKING SYSTEM.....	23
USER MANUAL	24
STEP BY STEP INSTRUCTIONS FOR USING THE SYSTEM	24
HOW TO RUN THE APPLICATION.....	25
HOW TO SET UP THE DATABASE.....	26
COMMON ERRORS AND SOLUTIONS	27
CHALLENGES AND REFLECTIONS	27
CHALLENGES FACED DURING DEVELOPMENT	27
HOW CHALLENGES WERE RESOLVED	28
LESSONS LEARNED	28
CONTRIBUTION STATEMENTS.....	28

INTRODUCTION

Purpose and scope of the system

The purpose of our banking system is to provide reliable, secure and proficient platform for seamless financial transactions and customer or user banking activities, our system is designed to automate and modernize traditional banking processes, reduce manual labour and improve accuracy and efficiency. This java based system will allow users to perform essential banking activities namely: deposits, account creation, cash transfers and monitor transactionsthrough our user friendly platform.

Background and motivation

Banking systems have an imperative role in society they help individuals organize and manage finances in a secure and efficient manner. Dating back to the 80's traditional banking activities such as deposits, cash transfers ,withdrawals and account management were manually handled using paper records and physical processes which were easily misplaced and inaccurate or inconsistent prone to human error and time consuming In light of such constant occurring errors my team and I were motivated to build an efficient java based system to create a secure , efficient and reliable platform capable of handling/managing daily banking operations in a structured digital environment which also allows us to demonstrate our advanced software development skills such as database integration ,system design and programming.

OBJECTIVES OF A JAVA BASED BANKING SYSTEM

In this phase of the project, we discuss the key objectives of our banking system

OBJECTIVES:

TO MANAGE CUSTOMER ACCOUNTS EFFICIETLY: System will allow users to create, update, and manage bank accounts securely and more accurately than traditional banking systems

TO IMPROVE DATA ACCURACY AND SECURITY: The banking system is built to reduce human error while protecting customer information through authentication and secure data storage

TO AUTOMATE BANKING TRANSACTIONS: The banking system will perform transactions such as deposits, withdrawals and cash transfers quickly and reliably

TO SUPPORT DATABASE INTERGRATION: The application will connect with a relational database to store customer details, account information and transactions of data efficiently

TO ENHANCE USER EXPERIENCE: The system provides a user-friendly interface that makes banking operations easy to perform

TO IMPROVE OPERATIONAL EFFICIENCY IN BANKING SERVICES: The system will reduce manual paperwork and speed up daily banking operations

TO PROVIDE FAST ACCESS TO BANKING SERVICES: Users will be able to have fast access to accounts information and transactions in real-time

SYSTEM OVERVIEW

HIGH LEVEL DESCRIPTION OF THE BANKING SYSTEM

The java banking system is a software system designed to manage and automate the core banking operations on a digital platform, the system allows customers and bank staff members to perform essential banking activities such as creating accounts depositing and withdrawing money, transferring money, check balances and viewing transaction history, this system is created using Java language and is connected to a relational database for secure storage and management of customer transaction data. The banking system aims to improve overall efficiency reduce manual processes while enhancing data accuracy and providing secure access to banking services, it also includes authentication features that will protect sensitive customer information and ensure that only authorized personnel can access the system.

LIST OF CORE FUNCTIONALITIES

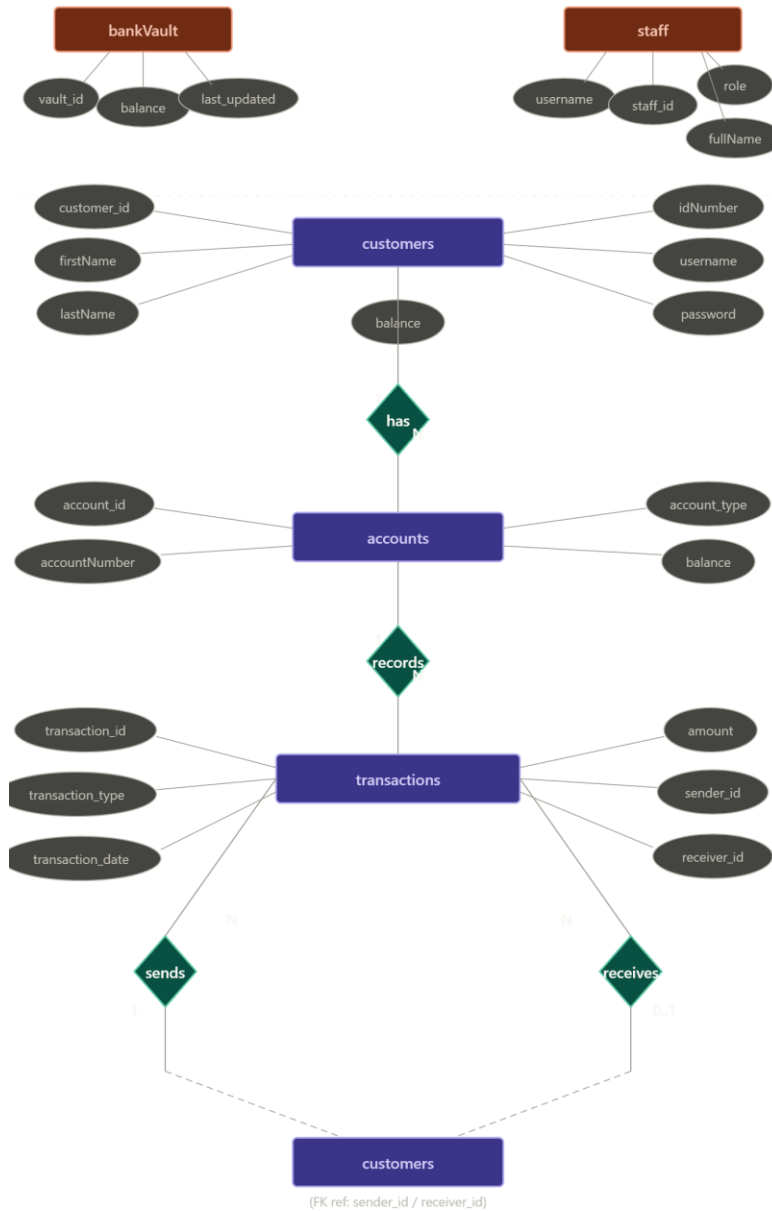
- USER REGISTRATION AND LOGIN - Allows customer and administration to create accounts and securely log into the system
- Account creation and management - Enables the creation, updating and deletion of customer bank accounts
- Balance inquiry - allows users to check the current account balance
- Transfer functionality - Allows users to transfer money from their account into another account.
- Deposit functionality - Enables users to deposit money into their accounts
- Withdrawal functionality - Allows users to withdraw money while checking for sufficient balance
- Database Management - connects the system to a relational database for storing customer accounts, and transaction information

TECHNOLOGIES AND TOOLS USED

- Programming Language: Java 26
- Database: Xampp
 - *Database query language: SQL*
- IDE: Visual Studio Code
- Application: NetBeans
- Version control: GitHub

DATABASE DESIGN

ENTITY RELATIONSHIP DIAGRAM (ERD)



EXPLANATION OF THE DATABASE TABLES.

- **Staff table:** The staff table stores information about all employees who use the banking system. It stores the staff login credentials, employee name, roles and permissions and staff balance or salaries.
- **Customer table:** The customers table stores information about all bank customers. It stores customers personal information, login details customers balances and customer ID numbers.
- **Accounts table:** The accounts table stores all bank account information. What is stored in the accounts table is account numbers, accounts balances, account types and customer ownership.
- **Bank Vault Table:** The bank Vault table stores the banks total available reserve funds. It stores total vault balance and last update time.
- **Transactions Table:** The transactions table stores all banking activities performed by customers. It stores all the deposits, withdrawals, transfers and transaction history.

PRIMARY AND FOREIGN KEY JUSTIFICATION.

1. Staff table

→PRIMARY KEY (staff _ id INT AUTO_INCREMENT PRIMARY KEY)

- Staff _ id uniquely identifies every staff member.
- Staff names and surnames may change but the ID remains the same .
- AUTO_INCREMENT automatically generates unique IDs.

2. Customer table

→PRIMARY KEY (customer _ id INT AUTO_INCREMENT PRIMARY KEY)

- Each customer needs a unique identifier.
- Customer can share names so names are not reliable identifiers.
- AUTO_INCREMENT prevents duplicate IDs.

3. Accounts table

→ PRIMARY KEY (account _ id INT AUTO_INCEMENT PRIMARY KEY)

- Every account needs a unique identifier.
- Customers may own multiple accounts.

- Easier to reference account in transactions.

→ FOREIGN KEY (FOREIGN KEY (customer _ id) REFERENCES customers (customer _ id))

- Connects each account to a valid customer.
- Prevents creating accounts for non-existing customers.
- Maintains integrity.

4. Bank Vault table

→ PRIMARY KEY (vault _ id INT AUTO_INCREMENT PRIMARY KEY)

- Identifies each vault record uniquely.
- Allows future expansion for multiple vault records or history tracking.

5. Transactions table

→ PRIMARY KEY (transaction _ id INT AUTO_INCREMENT PRIMARY KEY)

- Every transaction must be uniquely identifiable.
- Essential for:
 - auditing
 - banking history
 - fraud prevention
 - transaction tracing

→ FOREIGN KEY (FOREIGN KEY (Sender _ id) REFERENCES customers (customer _ id))

- Identifies which customer initiated the transaction.
- Ensures the sender exists in the customers table.
- Helps maintain transaction history integrity.

→ FOREIGN KEY (FOREIGN KEY (account _ id) REFERENCES accounts (account _ id))

- Identifies the customer receiving money during transfers.
- Ensures receivers are valid customers.
- Allows NULL because withdrawals and deposits may not have receivers.

NORMALIZATION EXPLAINATON

Our database follows the Third Normal Form (3NF) – that is, it is also in both 1NF and 2NF. The database is in First Normal Form (1NF) because every column stores a single atomic value and every table has a primary key, *for example customer_id, staff_id, and transaction_id*. The database is also in Second Normal Form (2NF) because all tables use single column primary keys, meaning every non-key column fully depends on the whole primary key with no partial dependencies. It reaches Third Normal Form (3NF) because no non-key column depends on another non-key column - *for example, instead of storing customer names and balances inside the transactions table, you store sender_id and receiver_id as foreign keys that reference back to the customers table*, meaning every column in every table depends only on its own primary key and nothing else, which eliminates all transitive dependencies and ensures data remains consistent across the entire banking system.

APPLICATION FEATURES DOCUMENTATION

FEATURE LIST WITH DESCRIPTION

1. Staff Management

- This feature allows the bank to manage employees who use the system such as administrators, tellers, branch managers, and loan officers.

2. Customer Registration

- Allows new customers to register and create profiles in the banking system. It also captures customers personal details, stores ID numbers and creates usernames and passwords to prevent duplicate registrations.

3. Customer Login Authentication

- Provides secure login access for customers using usernames and passwords like username verification and password authentication it also prevents unauthorized access and secure customer accounts.

4. Staff Login Authentication

- Allows staff members to securely access the banking system based on their role.

5. Account Creation

- Enables the creation of bank accounts for customers.

6. Multiple Account Support

- Allows one customer to own more than one account.

7. Deposit Feature

- Allows money to be deposited into customer accounts.

8. Withdrawal Feature

- Allows customers to withdraw money from their accounts.

9. Money Transfer Feature

- Allows customers to transfer money between accounts.

10. Transaction History

- Stores all banking activities performed in the system.

11. Bank Vault Management

- Represents the banks total available funds. It stores total bank balance, updates bank reserve amount and track financial status.

12. Balance Management

- Automatically updates balance after every transaction. it increases balance after deposit, decreases balance after withdrawals and synchronizes customer and account balances.

13. Role Based Access Control

- Different staff members have different permissions in the system.

14. Unique Account Number Generation

- Ensures every bank account has a unique account number. It also prevents duplicate accounts and simplifies account identification.

15. Database Security Constraints

- Protects data integrity within the system.

16. Audit and Tracking

- Allows the bank to monitor all activities performed in the system.

17. Automatic Timestamp Recording

- Automatically stores the data and time of transactions like transaction timestamp, activity tracking and banking history.

18. Data Integrity Enforcement

- It ensures accurate and valid relationship between tables and it also prevents invalid customer reference, orphan accounts and maintains consistent data.

Screenshots of running application

Output - **ASSIGNMENT** (run) #2

```
run:

*****
      WELCOME TO THE BANK
*****
Enter 1 if you are a staff member
Enter 2 if you are a customer
Enter 0 to exit
2

*****CUSTOMER LOGIN*****
Enter your username
fff
Enter your password
uuy
INVALID DETAILS, TRY AGAIN

*****
      WELCOME TO THE BANK
*****
Enter 1 if you are a staff member
Enter 2 if you are a customer
Enter 0 to exit
|
```

Figure 1: Incorrect details entered

```
Output
ASSIGNMENT (run) #2 × ASSIGNMENT (run) #3
run:
*****
WELCOME TO THE BANK
*****
Enter 1 if you are a staff member
Enter 2 if you are a customer
Enter 0 to exit
1

*****STAFF LOGIN*****
Enter your username
staff.ledi
Enter your password
staff778
LOGIN SUCCESSFUL

-----STAFF MENU-----
Enter 1 to Deposit
Enter 2 to Withdraw
Enter 3 to Transfer
Enter 4 to Check Bank Balance
Enter 5 to View All Customers
Enter 0 to return to Main Menu
|
```

Figure 2: Successful staff login

```
run:

*****
      WELCOME TO THE BANK
*****

Enter 1 if you are a staff member
Enter 2 if you are a customer
Enter 0 to exit
2

*****CUSTOMER LOGIN*****
Enter your username
thabo.dlamini
Enter your password
Pass@1234
LOGIN SUCCESSFUL. Welcome, Thabo!

-----CUSTOMER MENU-----
Enter 1 to register
Enter 2 to view your details
Enter 3 to update your details
Enter 4 to deposit money
Enter 5 to withdraw money
Enter 6 to transfer money
Enter 7 to check your balance
Enter 0 to return to the main menu
```

Figure 3: Successful customer login



```
-----CUSTOMER MENU-----
Enter 1 to register
Enter 2 to view your details
Enter 3 to update your details
Enter 4 to deposit money
Enter 5 to withdraw money
Enter 6 to transfer money
Enter 7 to check your balance
Enter 0 to return to the main menu
1
-----REGISTER-----
Enter your first name
ledi
Enter your last name
maluks
Enter your ID number
1234567890
Enter your email address
ledi@gmail.com
Enter your phone number
0987654321
Enter your username
1.maluks
Enter your password
445566
REGISTRATION SUCCESSFUL
Name          : ledi maluks
ID Number     : 1234567890
Email         : ledi@gmail.com
Phone        : 0987654321
Username      : 1.maluks
Account Number: 424740
Password      : *****

-----CUSTOMER MENU-----
Enter 1 to register
Enter 2 to view your details
Enter 3 to update your details
Enter 4 to deposit money
Enter 5 to withdraw money
Enter 6 to transfer money
Enter 7 to check your balance
Enter 0 to return to the main menu
|
```

Figure 4: Successful registration


```
-----CUSTOMER MENU-----
Enter 1 to register
Enter 2 to view your details
Enter 3 to update your details
Enter 4 to deposit money
Enter 5 to withdraw money
Enter 6 to transfer money
Enter 7 to check your balance
Enter 0 to return to the main menu
2
-----VIEW DETAILS-----
Name       : Thabo Dlamini
Username    : thabo.dlamini
Account Number : 100001
Balance     : R15420.5
Email       : thabo@email.com
Phone       : 0711234567

-----CUSTOMER MENU-----
Enter 1 to register
Enter 2 to view your details
Enter 3 to update your details
Enter 4 to deposit money
Enter 5 to withdraw money
Enter 6 to transfer money
Enter 7 to check your balance
Enter 0 to return to the main menu
|
```

Figure 5: View Details

```
-----CUSTOMER MENU-----
Enter 1 to register
Enter 2 to view your details
Enter 3 to update your details
Enter 4 to deposit money
Enter 5 to withdraw money
Enter 6 to transfer money
Enter 7 to check your balance
Enter 0 to return to the main menu
3
-----UPDATE DETAILS-----
Enter 1 to update username
Enter 2 to update password
Enter 3 to update phone number
Enter 4 to update email
1
Enter your new username
ThaboM
Username updated to: ThaboM

-----CUSTOMER MENU-----
Enter 1 to register
Enter 2 to view your details
Enter 3 to update your details
Enter 4 to deposit money
Enter 5 to withdraw money
Enter 6 to transfer money
Enter 7 to check your balance
Enter 0 to return to the main menu
|
```

Figure 6: Update Details

```
-----CUSTOMER MENU-----
Enter 1 to register
Enter 2 to view your details
Enter 3 to update your details
Enter 4 to deposit money
Enter 5 to withdraw money
Enter 6 to transfer money
Enter 7 to check your balance
Enter 0 to return to the main menu
4

-----DEPOSIT-----
Enter the account number
918273645
Enter the amount you want to deposit
700
Deposit successful.
Account Number   : 918273645
Amount Deposited: R700.0
New Balance      : R16120.5

-----CUSTOMER MENU-----
Enter 1 to register
Enter 2 to view your details
Enter 3 to update your details
Enter 4 to deposit money
Enter 5 to withdraw money
Enter 6 to transfer money
Enter 7 to check your balance
Enter 0 to return to the main menu
|
```

Figure 7: Successful Deposit

```
-----CUSTOMER MENU-----
Enter 1 to register
Enter 2 to view your details
Enter 3 to update your details
Enter 4 to deposit money
Enter 5 to withdraw money
Enter 6 to transfer money
Enter 7 to check your balance
Enter 0 to return to the main menu
5

-----WITHDRAWAL-----
Enter the account number
918273645
Enter the amount you want to withdraw
688

-----
Withdrawal successful.
Account Number   : 918273645
Amount Withdrawn : R688.0
Remaining Balance: R14732.5

-----CUSTOMER MENU-----
Enter 1 to register
Enter 2 to view your details
Enter 3 to update your details
Enter 4 to deposit money
Enter 5 to withdraw money
Enter 6 to transfer money
Enter 7 to check your balance
Enter 0 to return to the main menu
||
```

Figure 8: Successful withdrawal

```
-----CUSTOMER MENU-----
Enter 1 to register
Enter 2 to view your details
Enter 3 to update your details
Enter 4 to deposit money
Enter 5 to withdraw money
Enter 6 to transfer money
Enter 7 to check your balance
Enter 0 to return to the main menu
6

-----TRANSFER-----
Enter the account you want to transfer to
675
Enter the amount you want to transfer
8888
Transfer successful.
To Account Number : 675
Amount Transferred: R8888.0
Remaining Balance : R6532.5

-----CUSTOMER MENU-----
Enter 1 to register
Enter 2 to view your details
Enter 3 to update your details
Enter 4 to deposit money
Enter 5 to withdraw money
Enter 6 to transfer money
Enter 7 to check your balance
Enter 0 to return to the main menu
```

Figure 9: Successful Transfer

```
-----CUSTOMER MENU-----
Enter 1 to register
Enter 2 to view your details
Enter 3 to update your details
Enter 4 to deposit money
Enter 5 to withdraw money
Enter 6 to transfer money
Enter 7 to check your balance
Enter 0 to return to the main menu
2
-----VIEW DETAILS-----
Name       : Thabo Dlamini
Username   : thabo.dlamini
Account Number : 100001
Balance    : R15420.5
Email      : thabo@email.com
Phone      : 0711234567

-----CUSTOMER MENU-----
Enter 1 to register
Enter 2 to view your details
Enter 3 to update your details
Enter 4 to deposit money
Enter 5 to withdraw money
Enter 6 to transfer money
Enter 7 to check your balance
Enter 0 to return to the main menu
|
```

Figure 10: Customer View Details

```
-----CUSTOMER MENU-----
Enter 1 to register
Enter 2 to view your details
Enter 3 to update your details
Enter 4 to deposit money
Enter 5 to withdraw money
Enter 6 to transfer money
Enter 7 to check your balance
Enter 0 to return to the main menu
0

*****
WELCOME TO THE BANK
*****

Enter 1 if you are a staff member
Enter 2 if you are a customer
Enter 0 to exit
||
```

Figure 11:Returning to main menu (exit feature)

INPUT/OUTPUT DESCRIPTIONS

1. Successful staff member or customer login

- For staff members, the system only accepts usernames and passwords that contains the word *staff*, then the system will display the staff menu. For customers – the system accepts only usernames that contain their first name and a password that contains the @ symbol, then the system will display the customer menu.

2. Successful registration

- The user is asked what they would like to do. The user selected 1 to register, they enter all details required correctly, the system displays the information as well as “*Registration successful*”, the system proceeds to show the customer menu for further actions and services.

3. View details

- The user is asked what they would like to do. The user selected 2 to view their details, the system displays all their details and proceeds to show the customer menu for further actions and services.

4. Update Details

- The user is asked what they would like to do. The user selected option 3 to update their details. The user is then asked what information they are updating (in this case option 1 – to update the username), the user then enters the new/updated username. The system then displays the updated username and proceeds to show the customer menu for further actions and services.

5. Successful Deposit

- The user is asked what they would like to do. They selected 4 – to deposit money. The user is then asked to enter the account number and amount they would like to deposit. The system then displays “*Deposit successful*”, displays the amount deposited and the available balance (the new balance) and proceeds to show the customer menu for further actions and services.

6. Successful Withdrawal

- The user is asked what they would like to do. They selected 5 – to withdraw money. The user is then asked to enter the account number and amount they would like to withdraw. The system then displays “*withdrawal successful*”, displays the amount deposited and the available balance (the new balance) and proceeds to show the customer menu for further actions and services.

7. Successful Transfer

- The user is asked what they would like to do. They selected 6 – to transfer money. The user is then asked to enter the account number and amount they would like to transfer. The system then displays “*transfer successful*”, displays the amount transferred and the available balance (the new balance), and proceeds to show the customer menu for further actions and services.

8. Return to main menu (exit)

- The user is asked what they would like to do. They selected 0 – to return to the main menu. The system reverts back to the main menu and displays it.

ERROR HANDLING IN THE BANKING SYSTEM

1. Invalid Login Credentials - Occurs when a user enters the wrong username or password.

- The system checks the entered credentials against the database.
- If the username or password is incorrect:
 - access is denied
 - an error message is displayed
 - The user is prompted to try again.

2. Duplicate Username Error - Occurs when a user tries to register with an existing username.

- The system checks if the username already exists before saving.
- Registration is rejected if the username is already taken.

3. Duplicate ID Number Error - Occurs when a customer registers using an ID number that is already in the system.

- Duplicate ID numbers are blocked automatically by the database.

4. Insufficient Balance Error - Occurs when the customer tries to withdraw or transfer more money than available.

- The system checks the account balance before processing.
- if funds are insufficient:
 - Transactions are cancelled.
 - Balance remains unchanged.

5. Invalid Account Number - Occurs when money is transferred to non-existing account.

- The system searches for the account number in the database.
- If the account does not exist:
 - Transfer is rejected.
 - No money is deducted.

USER MANUAL

STEP BY STEP INSTRUCTIONS FOR USING THE SYSTEM

Requirements: java development kit, MySQL sever, MySQL work, java IDE

STEP 1: CLICK START MENU ON YOUR PC

1. Click the start Menu
2. Search for MySQL workbench
3. Open the application

STEP 2: CONNECT TO MYSQL

1. Click your local MySQL connection
2. Enter your MySQL password if prompted

STEP 3: IMPORT THE SQL FILE

1. At the top left corner, Click file
2. Select open SQL script or run SQL Script

3. Browse to the banking System project folder
4. Select the provided .SQL FILE
5. Click open

STEP 4: RUN THE SQL SCRIPT

1. Click run button at the top of the MySQL.
2. Wait for the script to finish running
3. The database and all required tables will be created automatically

STEP 5: VERIFY THE DATA BASE

1. On the left panel under schema, refresh the list
2. Confirm that the banking database appears
3. Expand the database to see tables: customer, account, transactions, transfer

HOW TO RUN THE APPLICATION

STEP 1: OPEN THE PROJECT FOLDER

- Find the banking system folder on your computer
- The folder must contain: java files, MySQL connector.jar,source code files

STEP 2. OPEN COMMAND TERMINAL (windows)

- Press windows +R
- Type: CMD
- Press enter

STEP 3. GO TO THE PROJECT FOLDER

- Type: CD Banking system
- Press enter

STEP 4. COMPILE THE JAVA FILES

- Type :javac-cp . MySQL-connector .jar SRC.java
- Press enter

STEP 5. RUN THE APPLICATION

- Type: java CP. mysql-connector.jar Main
- Press enter

STEP 6. USE THE SYSTEM

- Login using your username and password
- Select options from the menu: deposit, withdraw, transfer etc.

STEP 7. EXIT THE SYSTEM

- Logout
- Close the program window

HOW TO SET UP THE DATABASE

Step 1

- Install MySQL

Step 2

- Open MySQL Workbench and Connect

Step 3

- Create the Database (USING SQL QUERIES)

Step 4

- Create the Tables (USING SQL QUERIES)

Step 5

- Insert Test Data (USING SQL QUERIES)

Step 6

- Connect Java to the Database

Step 7

- Test the connection

Step 8

- Replace ArrayList with Database Calls

COMMON ERRORS AND SOLUTIONS

ERRORS	CAUSE	SOLUTIONS
Insufficient funds	Withdrawal or transfer amount exceeds the Balance	User must deposit more money before attempting the transaction again
Syntax error in SQL	Mistake in SQL script	Check and correct the SQL syntax before running the script
Access denied for user	Incorrect Database username or password	Update the database username and password in the java configuration file
Connection timeout	Database connection failed	Check MySQL server is running as well as the Internet settings are correct

CHALLENGES AND REFLECTIONS

CHALLENGES FACED DURING DEVELOPMENT

1. **The User interface design** - Creating a user-friendly system to accommodate both technical and non technical users while keeping the design simple and functional using Java was challenging.
2. **Input validation and error handling** - Ensuring that all user inputs were valid and handling unexpected errors without crashing the system required a lot of careful planning before implementation

HOW CHALLENGES WERE RESOLVED

1. **User interface solution** - We used a clean and consistent User interface design using proper layout managers user feedback messages and validations
2. **Error handling solution** - Our system checks User inputs to make sure correct information is entered if an error happened the program displays a message instead of crashing, this makes the system easier to use

LESSONS LEARNED

1. **The importance of planning before execution** - A properly and well thought out plan system design before coding can save a good amount of time and reduces errors during development phase
2. **Understanding user needs** - A clear understanding of user requirements can lead to better features and more improved usability
3. What we would differently is spend more time on planning and discussions analyzing requirements and designing the system before implementation, implement more comprehensive error handling and provide clearer user-friendly messages.

CONTRIBUTION STATEMENTS

I **Buhlebamaqwabe Gumede** was responsible for the design and implementation of the SQL database component of the Banking system project. Designing the relational database structure of the banking system. Creating the SQL tables of staff, customer, accounts, transaction and bank vault. Defining primary and foreign keys to establish relationships between tables and maintain referential integrity. Implementing database constraints such as PRIMARY KEY , FOREIGN KEY , UNIQUE , NOT NULL, and DEFAULT values. Developing SQL script for database creation and setup. Testing database functionality to ensure accurate data storage and retrieval. Assisting with the integration of the database with the Java application.

I **Theledi Ramouna Maluleka** contributed to the design, development, and testing of the Java banking system project. My responsibilities included creating and implementing the main features of the system, such as customer and staff login, customer registration, balance checking, deposits, withdrawals, money transfer functionalities and authentication of customers and staff members. I also designed the menu-driven interface to ensure smooth navigation between the different sections of the program. I further participated in debugging and improving the logical flow of the system by identifying and correcting errors related to user input, account validation, and transaction processing. I also helped in the creation of the SELECT, DELETE and UPDATE SQL queries which were required as well as compiling the documentation.

I **Tshepiso Leepile** was primarily responsible for the documentation and written technical components of the java-based banking system, my contribution was focused in ensuring that the documentation was professionally done and easy to understand for both technical and non-technical users.